

OOPS LAB MANUAL

Department of Artificial Intelligence and Data Science

Experiment: Inheritance, Encapsulation and Runtime Polymorphism

Concept Covered

- Hierarchical Inheritance
- Method Overriding
- Use of super
- Runtime Polymorphism
- Array of Base Class References

Scenario

An online shopping platform calculates order totals differently based on order type.

Order Types

1. Regular Order
2. Prime Order
3. International Order

Rules

- Regular → ₹200 shipping
 - Prime → No shipping
 - International → ₹200 shipping + ₹1000 customs
 - Must demonstrate runtime polymorphism
-

Given Bugged Base Class

```
class Order {

    public int orderId;
    private String customerName;
    private double itemPrice;

    public Order(int id, String name, double price) {
        orderId = id;
        name = customerName;
        itemPrice = price;
    }

    public String getCustomerName() {
        return customerName;
    }

    public double getPrice(double extra) {
        return itemPrice + extra;
    }

    public static double calculateTotal() {
        return 200;
    }

    public void display() {
        System.out.println("Order ID: " + orderId);
        System.out.println("Customer: " + customerName);
        System.out.println("Total Bill: " + calculateTotal());
    }
}
```

Tasks

- **Fix base class**
 - **Implement PrimeOrder**
 - **Implement InternationalOrder**
 - **Demonstrate runtime polymorphism**
-

Expected Output

Order ID: 101

Customer: Ananya

Total Bill: 2200.0

Order ID: 102

Customer: Rahul

Total Bill: 3000.0

Order ID: 103

Customer: Meera

Total Bill: 5200.0

Analytical Questions

1. **Why does static method break runtime polymorphism?**
 2. **What type of binding occurs in overridden methods?**
 3. **How does the code make sure abstraction is retained?**
-

EXERCISE 2: Smart Vehicle Toll Collection System

Scenario

Highway toll based on vehicle type.

Vehicle Types:

- Car – ₹100
 - Truck – ₹300
 - ElectricCar – ₹50
 - Use runtime polymorphism and inheritance, constructors.
-

Expected Output

Vehicle: KA01AB1234

Toll: 100.0

Vehicle: KA05CD5678

Toll: 300.0

Vehicle: KA09EF9999

Toll: 50.0

Analytical Questions

- 1. In the Vehicle–Car example, overridden methods are resolved at runtime, but variables are resolved at compile time. Demonstrate this using a small code snippet and explain the difference.**
 - 2. What happens if calculateToll() is static?**
-

EXERCISE 3: Smart University Course Fee System

Scenario

Program Types:

- Undergraduate → +5000
- Postgraduate → +10000
- PhD → -8000

Use runtime polymorphism and inheritance, constructors

Expected Output

Student: Arjun

Total Fee: 55000.0

Student: Sneha

Total Fee: 70000.0

Student: Vikram

Total Fee: 62000.0

Analytical Questions

1. Why must method signatures match in overriding?
 2. Can constructors be overridden?
-

EXERCISE 4: Smart Digital Payment Processing System (Advanced)

Scenario

Payment Types:

- CreditCardPayment
- UpiPayment
- CryptoPayment

Rules:

- Credit Card → 2% + ₹500 if >50000
- UPI → 1% discount if >100000
- Crypto → 5% fee

Requirements:

- Abstract base class
- Constructor chaining
- Override calculateFinalAmount()
- Override toString()

- Use array for polymorphism
-

Expected Output

Transaction ID: TXN101

Payer: Arun

Final Amount: 61700.0

Transaction ID: TXN102

Payer: Megha

Final Amount: 118800.0

Transaction ID: TXN103

Payer: Rohit

Final Amount: 52500.0

Analytical Questions

- 1. Why must Payment be abstract?**
 - 2. Can abstract classes have constructors?**
 - 3. What happens if overridden method is static?**
-

EXERCISE 5: Smart Library Fine Management System (Debug + Design)

Scenario

Library fine system for different member types:

- Student → ₹5/day + ₹200 if days > 30
- Faculty → ₹2/day
- Guest → ₹10/day + ₹500 if days > 15

You must debug base class and design subclasses.

Given Bugged Base Class

```
class LibraryMember {  
  
    public int memberId;  
    private String memberName;  
    private int numberOfDaysLate;  
  
    public LibraryMember(int id, String name, int days) {  
        memberId = id;  
        name = memberName;  
        numberOfDaysLate = days;  
    }  
  
    public int getDaysLate(int extra) {  
        return numberOfDaysLate + extra;  
    }  
  
    public static double calculateFine() {  
        return numberOfDaysLate * 2;  
    }  
  
    public void displayFine() {  
        System.out.println("Member ID: " + memberId);  
        System.out.println("Name: " + memberName);  
        System.out.println("Fine: " + calculateFine());  
    }  
}
```

Student Tasks

Part A – Debug Base Class

- **Fix compile-time errors**
- **Fix constructor mapping**
- **Fix encapsulation**
- **Fix static binding issue**

Part B – Design System

- **Create Student**
 - **Create Faculty**
 - **Create Guest**
 - **Use base class reference array**
 - **Demonstrate runtime polymorphism**
-

Expected Output

Member ID: 1

Name: Arjun

Fine: 400.0

Member ID: 2

Name: Meera

Fine: 40.0

Member ID: 3

Name: Rahul

Fine: 700.0

Analytical Questions

- 1. Why does static method prevent dynamic binding?**
- 2. What is constructor shadowing?**
- 3. Can variables be overridden in Java?**
- 4. Where does runtime polymorphism occur?**

Appendix:

Code Example

// Polymorphism with Hierarchical Inheritance

// Overtime added inside Developer bonus

```
public class Employeep {

    private String name;
    protected double salary;

    Employeep(String n, double s) {
        this.name = n;
        this.salary = s;
    }

    void calculateSalary() {
        System.out.println("Employee Salary: " + salary);
    }

    public double bonus(float percent, int hours) {
        return (percent / 100.0) * salary;
    }
}

// Manager class
```

```
class Managerp extends Employeep {
```

```
    Managerp(String n, double s) {
```

```
        super(n, s);
```

```
    }
```

```
    public double bonus(float percent, int hours) {
```

```
        return 1.5 * super.bonus(percent, hours);
```

```
    }
```

```
}
```

```
// Developer class (NOT a Manager)
```

```
class Developerp extends Employeep {
```

```
    Developerp(String n, double s) {
```

```
        super(n, s);
```

```
    }
```

```
    public double bonus(float percent, int hours) {
```

```
        double overtimePay = 500 * hours;
```

```
        return super.bonus(percent, hours) + overtimePay;
```

```
    }
```

```
    public static void main(String[] args) {
```

```
        Employeep[] emparray = new Employeep[3];
```

//array of base class reference type initialisation

```
emparray[0] = new Employeep("Ravi", 10000);
emparray[1] = new Managerp("Anita", 150000);
emparray[2] = new Developerp("Kiran", 40000);

for (int i = 0; i < emparray.length; i++) {

    emparray[i].calculateSalary();
    System.out.println("Bonus: " + emparray[i].bonus(5.0f, 8));
}
}
```

Sample Question

1. Identify the type of inheritance used.
 2. Which method demonstrates runtime polymorphism?
 3. What is the role of super.bonus() in Managerp and Developerp?
 4. Why is salary declared as protected and not private?
 5. What will happen if bonus() is declared as static in base class?
-

Expected Output

For:

- Ravi → Salary 10000
- Anita → Salary 150000
- Kiran → Salary 40000
- Bonus percentage = 5%
- Overtime hours = 8

Calculation:

Ravi

5% of 10000 = **500**

Anita

5% of 150000 = 7500

Manager gets $1.5 \times 7500 = \mathbf{11250}$

Kiran

5% of 40000 = 2000

Overtime = $500 \times 8 = 4000$

Total = 6000